

Homework 1 - Custom MCP Math & Database Tool Server

One-page implementation and verification summary

Repository github.com/Sudashiii/dibse_industrial_computing

Stand: 06.09.2026

Implementierung

- SQLite inventory lookup with automatic database creation and seed data.
- Tiered discount engine with 0 / 5 / 10 / 15 percent volume tiers.
- JSONL audit tool plus audit://events MCP resource.
- MCP Stdio server and bounded ReAct agent via LiteLLM and OpenRouter.

Verifikation

- 37 passed in 2.62s
- compose config: valid
- Two live OpenRouter prompts are included below.
- Execution logs: litellm_run_prompt1.log and prompt2.log.

Live ReAct examples and tool observations

Live ReAct examples - OpenRouter

EXAMPLE 1 | logs\litellm_run_prompt1.log

PROMPT

Prüfe den Bestand des Industrial Sensor und berechne den Rabatt für 120 Stück. Nenne mir am Ende die wichtigsten Werte.

TOOL lookup_inventory -> stock=120 | price=49.90 EUR

TOOL calculate_tiered_discount -> discount=506.49 EUR
net=5481.51 EUR

ANSWER (condensed)

Bestand: 120 Stück; Rabatt: 506.49 EUR; Nettobetrag: 5481.51 EUR.

EXAMPLE 2 | logs\litellm_run_prompt2.log

PROMPT

Wie viele Safety Light Curtain sind auf Lager und reicht der Bestand für eine Bestellung von 5 Stück?

TOOL lookup_inventory -> stock=6 | price=1299.00 EUR

ANSWER (condensed)

Es sind 6 Stück auf Lager. Der Bestand reicht für eine Bestellung von 5 Stück.

Checks

```
Local verification

> .venv\Scripts\python.exe -m pytest -q
37 passed in 2.62s

> docker compose config --quiet
compose config: valid

> litellm_agent.py --help
litellm_agent.py: CLI available

Focused evidence view - timestamps and unrelated environment output omitted
```

Links: [README](#)

[LiteLLM config](#)

Relevant integration path



Focused screenshots contain no credentials or unrelated desktop content.